SOLIDITY VOTING PROJECT(TEMPLATE)

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

contract Vote {
    struct Voter {
        string name;
        uint256 age;
        uint256 voterId;
        Gender gender;
        uint256 voteCandidateId;
        address voterAddress;
    }

    struct Candidate {
        string name;
        string party;
        uint256 age;
        Gender gender;
        uint256 candidateId;
        address candidateAddress;
        uint256 votes;
    }

    address electionCommission;
    address public winner;
    uint256 nextVoterId = 1;
    uint256 nextCandidateId = 1;
    uint256 startTime;
    uint256 endTime;
    bool stopVoting;

    mapping(uint256 => Voter) voterDetails;
    mapping(uint256 => Candidate) candidateDetails;

    enum VotingStatus {
        NotStarted,
        InProgress,
```

```
        Ended
    }
    enum Gender {
        NotSpecified,
        Male,
        Female,
        Other
    }

    constructor() {}

    modifier isVotingOver() {
        _;
    }

    modifier onlyCommissioner() {
        _;
    }

    function registerCandidate(
        string calldata _name,
        string calldata _party,
        uint256 _age,
        Gender _gender
    ) external {}

    function isCandidateNotRegistered(address _person)
        internal
        view
        returns (bool)
    {}

    function getCandidateList() public view returns (Candidate[] memory)
{}

    function isVoterNotRegistered(address _person)
        internal
        view
        returns (bool)
    {}
```

```
    function registerVoter(
        string calldata _name,
        uint256 _age,
        Gender _gender
    ) external {}

    function getVoterList() public view returns (Voter[] memory) {}

    function castVote(uint256 _voterId, uint256 _id) external {}

    function setVotingPeriod(uint256 _startTime, uint256 _endTime)
        external
        onlyCommissioner
    {}

    function getVotingStatus() public view returns (VotingStatus) {}

    function announceVotingResult() external onlyCommissioner {}

    function emergencyStopVoting() public onlyCommissioner {}
}
```

SOLIDITY VOTING PROJECT BASIC COMPLETE

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

contract Vote {
    //first entity
    struct Voter {
        string name;
        uint256 age;
        uint256 voterId;
        Gender gender;
```

```
        uint256 voteCandidateId; //candidate id to whom the voter has
voted
        address voterAddress; //EOA of the voter
    }

    //second entity
    struct Candidate {
        string name;
        string party;
        uint256 age;
        Gender gender;
        uint256 candidateId;
        address candidateAddress; //candidate EOA
        uint256 votes; //number of votes
    }

    //third entity
    address public electionCommission;
    address public winner;
    uint256 nextVoterId = 1; //starting
    uint256 nextCandidateId = 1; //starting

    //voting period
    uint256 startTime;
    uint256 endTime;
    bool stopVoting;

    mapping(uint256 => Voter) voterDetails;
    mapping(uint256 => Candidate) candidateDetails;

    enum VotingStatus {
        NotStarted,
        InProgress,
        Ended
    }
    enum Gender {
        NotSpecified,
        Male,
        Female,
        Other
```

```
    }

    constructor() {
        electionCommission = msg.sender; //msg.sender is a global variable
    }

    modifier isVotingOver() {
        require(
            block.timestamp <= endTime && stopVoting == false,
            "Voting time is over"
        ); //user is modifier ko call kab karega uspe dependent hai
        _;
    }

    // end time - 1720540700 - Tuesday, 9 July 2024 21:28:20
    // current time - 1720540861 - Tuesday, 9 July 2024 21:31:01

    modifier onlyCommissioner() {
        require(msg.sender == electionCommission, "Not authuorized");
        _;
    }

    modifier isValidAge(uint256 _age) {
        require(_age >= 18, "not eligible for voting");
        _; //agar yeh nai hua to? then send this statement.
    }

    function isCandidateNotRegistered(address _person)
        private
        view
        returns (bool)
    {
        for (uint256 i = 1; i < nextCandidateId; i++) {
            if (candidateDetails[i].candidateAddress == _person) {
                return false;
            }
        }
        return true;
    }
```

```
    function isVoterNotRegistered(address _person) private view returns
(bool) {
        for (uint256 i = 1; i < nextVoterId; i++) {
            if (voterDetails[i].voterAddress == _person) {
                return false;
            }
        }
        return true;
    }

    function registerCandidate(
        string calldata _name,
        string calldata _party,
        uint256 _age,
        Gender _gender
    ) external isValidAge(_age) {
        require(
            isCandidateNotRegistered(msg.sender),
            "You are already registered"
        );
        require(nextCandidateId < 3, "Candidate Registration Full");
        require(
            msg.sender != electionCommission,
            "Election Commision not allowed to register"
        );

        candidateDetails[nextCandidateId] = Candidate({
            name: _name,
            party: _party,
            gender: _gender,
            age: _age,
            candidateId: nextCandidateId,
            candidateAddress: msg.sender,
            votes: 0
        });
        nextCandidateId++;
    }

    //MOST IMP
    function getCandidateList() public view returns (Candidate[] memory) {
```

```
        Candidate[] memory candidateList = new Candidate[](nextCandidateId
- 1); //initialize an empty array of length = `nextCandidateId - 1`
        for (uint256 i = 0; i < candidateList.length; i++) {
            candidateList[i] = candidateDetails[i + 1];
        }
        return candidateList;
    }

    function registerVoter(
        string calldata _name,
        uint256 _age,
        Gender _gender
    ) external isValidAge(_age) {
        require(isVoterNotRegistered(msg.sender), "You are already
registered");

        voterDetails[nextVoterId] = Voter({
            name: _name,
            age: _age,
            voterId: nextVoterId,
            gender: _gender,
            voteCandidateId: 0,
            voterAddress: msg.sender
        });
        nextVoterId++;
    }

    function getVoterList() public view returns (Voter[] memory) {
        uint256 lengthArr = nextVoterId - 1;
        Voter[] memory voterList = new Voter[](lengthArr);
        for (uint256 i = 0; i < voterList.length; i++) {
            voterList[i] = voterDetails[i + 1];
        }
        return voterList;
    }

    function castVote(uint256 _voterId, uint256 _candidateId)
        external
        isVotingOver
    {
```

```
        require(block.timestamp >= startTime, "Voting has not started
yet");
        require(
            voterDetails[_voterId].voteCandidateId == 0,
            "You have already voted"
        );
        require(
            voterDetails[_voterId].voterAddress == msg.sender,
            "You are not authourized"
        );
        require(
            _candidateId >= 1 && _candidateId < 3,
            "Candidate Id is not correct"
        );
        voterDetails[_voterId].voteCandidateId = _candidateId; //voting to
_candidateId
        candidateDetails[_candidateId].votes++; //increment _candidateId
votes
    }

    function setVotingPeriod(
        uint256 _startTimeDuration,
        uint256 _endTimeDuration
    ) external onlyCommissioner {
        require(
            _endTimeDuration > 3600,
            "_endTimeDuration must be greater than 1 hour"
        );
        startTime = 1720799550 + _startTimeDuration; //_startTimeDuration
= 3600 , _endTimeDuration = 3600
        endTime = startTime + _endTimeDuration;
    }

    // function setVotingPeriod(uint256 _startInSeconds)
    //     external
    //     onlyCommissioner
    // {
    //     startTime = block.timestamp + _startInSeconds; // Future start
time
```

```
    //      endTime = startTime + 3600; // 1 hour voting period after start
time
    // }
    //If you call setVotingPeriod(7200), voting will start 2 hours from
now and last for 1 hour.

    function getVotingStatus() public view returns (VotingStatus) {
        if (startTime == 0) {
            return VotingStatus.NotStarted;
        } else if (endTime > block.timestamp && stopVoting == false) {
            return VotingStatus.InProgress;
        } else {
            return VotingStatus.Ended;
        }
    }

    function announceVotingResult() external onlyCommissioner {
        uint256 max = 0;
        for (uint256 i = 1; i < nextCandidateId; i++) {
            if (candidateDetails[i].votes > max) {
                max = candidateDetails[i].votes;
                winner = candidateDetails[i].candidateAddress;
            }
        }
    }

    function emergencyStopVoting() public onlyCommissioner {
        stopVoting = true;
    }

    //if votingStatus==NotStarted then do this
    //else if votingStatus==Started then do that
    //else bla bla
}
```